Bioinformatics, Original Paper, Sequence analysis — draft

dinara-align: hardware-aware algorithms for exact pairwise alignment

Konstantinos Kyriakidis[1,*] and Benedict Paten[1,*]
[1]UC Santa Cruz Genomics Institute, Santa Cruz, CA, USA
[*]To whom correspondence should be addressed: kkyriaki@ucsc.edu, bpaten@ucsc.edu

Motivation: Exact pairwise alignment of long sequencing reads remains computationally demanding. Band doubling over bit-parallel blocks, the wavefront algorithm and A* with seed heuristics have greatly reduced the number of cells an exact aligner must compute. A large part of the remaining running time depends on how the computation maps onto the processor: loop-carried dependency chains, memory-bound table lookups, and bounds and thresholds that are poorly matched to the input or the hardware.
Results: We present dinara-align, an exact pairwise aligner built on algorithmic changes that address these bottlenecks: a direct construction of the neighbours of exact seed matches that provably preserves local pruning, diagonal transition vectorised across diagonals, band bounds accepted only when independent estimates agree, and per-pair optimality certificates for banded inter-sequence batches, together with a bit-parallel sweep arranged for instruction latency. On an Intel Skylake-X processor, dinara-align had the lowest mean running time among the exact aligners tested on all 22 datasets of the A*PA2 and QuickEd benchmarks, aligning ultra-long nanopore reads in 118 ms on average, compared with 283 ms for A*PA2-full, and was 6.7 times faster than QuickEd in the geometric mean. It was 1.7 to 3.8 times faster than WFA and BiWFA at gap-affine costs and computed global scores for 500,000 short-read pairs on one core 4.6 times faster than WFA2-lib.
Availability and implementation: dinara-align is free software under the Mozilla Public License 2.0, available at `https://github.com/kokyriakidis/dinara-align` with C, C++, Python and command-line interfaces.
Contact: kkyriaki@ucsc.edu, bpaten@ucsc.edu
Supplementary information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

Pairwise alignment determines the minimum-cost series of substitutions, insertions and deletions that transforms one sequence into another. The classical dynamic programming algorithm computes an $n \times m$ matrix (Gotoh, 1982), which is practical for short reads but not for the hundreds of kilobases spanned by current nanopore reads. Heuristic methods restrict the computed region and lose the guarantee of optimality; exact methods instead avoid only those cells that provably cannot lie on an optimal path.

Three ideas underlie current exact aligners. *Band doubling* computes only the diagonals that a path within a given cost bound can reach, and doubles the bound until the optimal cost lies within it (Ukkonen, 1985). *Bit-parallelism* encodes a column of 64 cells in one machine word (Myers, 1999), and Edlib combines it with band doubling (Šošić and Šikić, 2017). *Diagonal transition* extends the furthest-reaching paths one cost at a time, following runs of matches at no cost (Ukkonen, 1985; Myers, 1986); WFA generalised it to gap-affine costs (Marco-Sola et al., 2021) and BiWFA reduced its memory to linear in the distance (Marco-Sola et al., 2023). A*PA introduced A* search with a gap-chaining seed heuristic (Groot Koerkamp and Ivanov, 2024), and A*PA2 combined this heuristic with band doubling over SIMD blocks of bit-parallel words, aligning ultra-long reads up to 19 times faster than Edlib and BiWFA (Groot Koerkamp, 2024). A second line of work accelerates the full dynamic program on vector hardware, using anti-diagonal (Wozniak, 1997), striped (Farrar, 2007; Zhao et al., 2013) and prefix-scan (Daily, 2016) layouts, difference recurrences (Suzuki and Kasahara, 2018; Li, 2018), adaptive blocks (Liu and Steinegger, 2023) and inter-sequence parallelism (Rognes, 2011; Rahn et al., 2018); these methods compute the full matrix or a fixed band, so their running time grows with its size rather than with the distance.

In the exact methods, a large part of the remaining running time is determined by how the computation maps onto the processor. The seed heuristic is built by scalar lookups in tables larger than the caches. Diagonal transition extends one diagonal at a time. Band doubling spends whole rounds when its bound is far from the distance. Batches of short pairs are vectorised across pairs, but each pair's full matrix is computed. And the bit-parallel sweep carries a dependency from each column to the next, so its speed is set by the latency of that chain rather than by its number of operations. Here we present dinara-align, an exact aligner whose algorithms address these bottlenecks. Our contributions are:

1. **Direct neighbour construction for the seed heuristic.** The inexact neighbours of an exact seed match are inserted without search, and we prove that this preserves the outcome of local pruning (Lemma 1).
2. **Gathered diagonal transition.** Diagonal transition and the gap-affine wavefront extend eight diagonals per step using gather instructions.
3. **Band bounds from agreeing estimates.** A band-doubling bound is extrapolated from partial progress only when two independent estimates agree.
4. **Certified bands for inter-sequence batches.**

Each short pair is aligned in one vector lane within a narrow band whose optimality is certified per lane by a lower bound on the cost of any path that leaves it (Proposition 1), adapting the speculate-and-verify approach of the SeedEx accelerator (Fujiki et al., 2020) to software batches.

5. **A latency-aware bit-parallel sweep.** On AVX-512, two staggered groups of words are interleaved, extending the instruction-level parallelism of A*PA2, and two Boolean identities shorten the loop-carried chain of Myers' step; the two are worth 4.5 to 5% and 1 to 2% on the ultra-long reads, and 11.5% and 4% on 100 kbp pairs at 20% errors.

2 Methods

2.1 Problem definition and overview

Let $A = a_1 \dots a_n$ be the reference and $B = b_1 \dots b_m$ the query, and let cell (i, j) lie on diagonal $j - i$. Under gap-affine costs, a mismatch costs x and a gap of k letters costs $g(k) = o + ke$; unit costs correspond to $x = e = 1$ and $o = 0$, and two-piece costs take the minimum of two such functions (Li, 2018). A substitution matrix instead assigns each pair of letters a score to be maximised. We consider global, ends-free, extension and local alignment. An alignment is *exact* if it is optimal under the requested costs, mode, band and cost limit; among optimal alignments, a fixed rule places indels leftmost or rightmost, so that the result does not depend on which algorithm computed the cost.

Each problem is assigned to an algorithm (Table 1). For unit costs we follow A*PA2 (Groot Koerkamp, 2024): band doubling over tiles of 256 columns of bit-parallel words, pruned by the gap-chaining seed heuristic (Groot Koerkamp and Ivanov, 2024), with a diagonal-transition traceback inside each tile; near-identical pairs are aligned by diagonal transition alone. Gap-affine costs use the bidirectional wavefront of WFA and BiWFA (Marco-Sola et al., 2021, 2023). A short diagonal-transition search estimates the distance, and the estimate, the length and the cost model select the algorithm. The thresholds were set from measurements on each instruction set and are fixed at compile time; each was checked again by halving and doubling it (Supplementary Section S1).

2.2 A shorter critical path for Myers' recurrence

As in Edlib and A*PA2 (Šošić and Šikić, 2017; Groot Koerkamp, 2024), the query is packed 64 bases per word, and Myers' recurrence (Myers, 1999) advances a word by one column. With match mask E, vertical difference masks P_v, M_v and the incoming negative horizontal difference folded into $E' = E \vee M_h^{\text{in}}$, A*PA2 computes

$$\begin{aligned} c &= (E' \wedge P_v) + P_v, \\ X_h &= (c \oplus P_v) \vee E', \\ P_h &= M_v \vee \neg(X_h \vee P_v), \\ M_h &= P_v \wedge X_h. \end{aligned} \tag{1}$$

The horizontal masks P_h and M_h determine the vertical masks of the next column, so the path from c to them lies on the loop-carried chain. Since $(c \oplus P_v) \vee P_v = c \vee P_v$ and $P_v \wedge (c \oplus P_v) = P_v \wedge \neg c$, the same masks are obtained as

$$\begin{aligned} P_h &= M_v \vee \neg\big(c \vee (E' \vee P_v)\big), \\ M_h &= (P_v \wedge \neg c) \vee (P_v \wedge E'), \end{aligned} \tag{2}$$

where $E' \vee P_v$ and $P_v \wedge E'$ do not depend on c and are computed off the critical path. This removes two operations from the loop-carried chain and, for a single word, reduces the cost from 14 to 8 cycles per column on the Skylake-X. Myers (Myers, 1999), Edlib and A*PA2 use the original form, and we are not aware of a previous description of the regrouped one.

Words occupy the lanes of a vector, staggered by one column, so that the horizontal difference leaving one word enters the next through a lane rotation (Wozniak, 1997; Groot Koerkamp, 2024). Every column then waits for the rotation of the previous one, and A*PA2 processes two four-lane vectors together to overlap these waits. On AVX-512, where a vector holds eight words, dinara-align interleaves two eight-word groups, the lower lagging ten columns behind the upper, so that the instructions of each group fill the latency of the other; with narrower vectors a single group is used. Bands of two or three words, in which every operation lies on the critical path, are computed in general-purpose registers, whose operations have lower latency. On AVX-512 the sweep requires about 2.4 cycles per word and column.

2.3 Band bounds from agreeing estimates

Band doubling returns the exact distance for any initial bound, but its running time depends on how close that bound is to the distance. A partial computation that has reached cost s at anti-diagonal r suggests the extrapolated distance $\hat{d} = s(n + m)/r$. This extrapolation is reliable when edits are spread evenly along the pair and can be too high by orders of magnitude when they are concentrated, as near the ends of nanopore reads. dinara-align therefore computes two extrapolations, $\hat{d}_1$ from the first eight edits of a diagonal-transition search and $\hat{d}_2$ from the same search continued until its budget, and sets the first bound from them only if $\max(\hat{d}_1, \hat{d}_2) \le \frac{4}{3} \min(\hat{d}_1, \hat{d}_2)$. Otherwise, as in A*PA2, the bound starts 256 above the known lower bound and doubles. Within a round, the distance is extrapolated again after one eighth, one quarter and one half of the columns; the bound is lowered if the extrapolation lies below it, and the round is abandoned if the extrapolation exceeds it. Because these rules change only the sequence of bounds, the result remains exact.

Table 1: Algorithms in dinara-align and the changes introduced in this work.

Problem	Algorithm	Changes (section)
Unit costs, long pairs	Band doubling over bit-parallel tiles with the gap-chaining seed heuristic (A*PA2)	Regrouped recurrence, staggered groups (2.2); agreeing bounds (2.3); neighbour construction (2.5); traceback within a memory limit (S1)
Unit costs, similar pairs	Diagonal transition (WFA, BiWFA)	Gathered extension (2.4)
Gap-affine, two-piece	Bidirectional wavefront (WFA, BiWFA); full sweep in difference recurrences for divergent pairs (Suzuki and Kasahara)	Gathered extension (2.4); score-derived band (2.7); handover gated on work spent (S1)
Substitution matrices, local	Anti-diagonal vector sweep; end-first local alignment (SSW)	Backward wavefront from the best end (2.7)
Batches of short pairs	Inter-sequence vectorisation (SWIPE, SeqAn, parasail)	Certified per-lane bands (2.6)

2.4 Gathered diagonal transition

Diagonal transition extends each diagonal of a wavefront along its run of matches. dinara-align compares eight letters at a time by the exclusive-or of two 64-bit words and a count of trailing zeros. On AVX-512, eight diagonals are extended together: their letters are loaded with gather instructions, and the wavefront is padded with unreachable diagonals so that every step processes full vectors under branch-free masks. The gather masks are derived from the lanes of each group, which avoids a dependency between consecutive gathers that would otherwise serialise them. The same vectorised step serves near-identical pairs, computed from one or both ends (Marco-Sola et al., 2023), the traceback inside each tile, restricted to the diagonals that can still reach the traced cell, and the three-layer gap-affine wavefront.

2.5 Seed heuristic: direct neighbour construction

The reference is divided into disjoint seeds of 12 bases, matched exactly, or of 16 bases, matched with at most one edit (Groot Koerkamp and Ivanov, 2024). A 16-base seed that matches with at most one edit matches one of its two 8-base halves exactly, so each half indexes a table. Since these tables exceed the caches, the lookups for 16 query rows are issued together, so that their memory accesses overlap, and only seeds whose matches could still chain to the end of the pair are considered.

An exact match μ of a seed at query rows $[i, i+16)$ always has four neighbouring matches with one edit: two share its start, at rows $[i, i+15)$ and $[i, i+17)$, and two share its end, at $[i+1, i+16)$ and $[i-1, i+16)$. Because chaining is all-or-nothing, a neighbour one diagonal away may chain where μ cannot, and the neighbours are required for the heuristic to remain a lower bound. Rather than locating them by search and testing each for local pruning, dinara-align inserts them whenever μ is retained, which is justified by the following lemma. Local pruning retains a match ν of a seed only if $c(\nu) + h(e(\nu)) < \tau$, where $c(\nu)$ is its cost, $e(\nu)$ its end cell, $h(u)$ the least cost of a path from u across the following seeds, and τ a threshold that depends only on the seeds.

Lemma 1. If an exact match μ fails local pruning, so does each of its four neighbours.

Proof. A neighbour ν that shares the end of μ has $e(\nu) = e(\mu)$ and $c(\nu) = 1 > c(\mu) = 0$. A neighbour that shares the start ends on an adjacent diagonal, and one insertion or deletion from $e(\mu)$ reaches a cell on that diagonal at or beyond $e(\nu)$. The least remaining cost does not increase along a diagonal (Ukkonen, 1985), so $h(e(\mu)) \le 1 + h(e(\nu))$ and $c(\mu) + h(e(\mu)) \le c(\nu) + h(e(\nu))$. In both cases the pruning test value of ν is at least that of μ. □

Neighbours sharing the start therefore raise the layer of that start by the chain length available from their ends, and neighbours sharing the end are placed one layer below it, without a search or a pruning test of their own.

2.6 Certified bands for inter-sequence batches

Batches of short pairs are aligned with one pair per vector lane (Rognes, 2011; Rahn et al., 2018), first in saturating 8-bit lanes and, for pairs that saturate, in 16-bit lanes (Zhao et al., 2013; Daily, 2016). Rather than computing each matrix in full, a lane computes the band of diagonals $[d_\ell, d_h]$ containing the main diagonal 0 and the end diagonal $\delta = m - n$, together with one further diagonal on either side. Its optimality is certified with the following bound, a cost form of the out-of-band bounds of Gibrat (2018) and Fujiki et al. (2020), stated for gap-affine and two-piece costs and with a strict form for tie-breaking.

Proposition 1. Let C be the least cost of an alignment within $[d_\ell, d_h]$, and let $L(d) = g(|d|) + g(|d - \delta|)$. If $C \le \min\big(L(d_h + 1), L(d_\ell - 1)\big)$, then C is the optimal cost; if the inequality is strict, every optimal alignment lies within the band.

Proof. An alignment that leaves the band visits diagonal $d_h + 1$ or $d_\ell - 1$. Consider $d = d_h + 1 > \max(0, \delta)$; the case below the band is symmetric. Starting on diagonal 0, the alignment contains insertions totalling at least d letters to reach d, and deletions totalling at least $d - \delta$ letters to end on δ. These form at least one gap of each kind, and since g is subadditive (for two-piece costs, with the usual convention that the longer piece has the lower extension cost), their total cost is at least $g(d) + g(d - \delta) = L(d)$. Every alignment outside the band therefore costs at least $\min\big(L(d_h + 1), L(d_\ell - 1)\big)$. □

Bounds of this kind underlie band doubling (Ukkonen, 1985) and have been used to verify narrow-band results in software (Gibrat, 2018) and in hardware (Fujiki et al., 2020); here the certificate is evaluated independently in each vector lane of an inter-sequence batch. For alignments, the strict form guarantees that the alignment selected by the tie rule lies within the band. Pairs that fail the certificate, about one in seven in our short-read benchmark, are recomputed over every diagonal that a cheaper path could visit, grouped by the width of that band so that the lanes of a group finish together.

2.7 Gap-affine and score-based alignment

A scoring scheme with match score a, mismatch score b and gap scores $-(o + ke)$ can be converted into costs for global alignment (Eizenga and Paten, 2022). Every alignment covers $n + m$ letters, so its score S and a gap-affine cost C satisfy

$$\begin{aligned} S &= \tfrac{1}{2}\big(a(n+m) - C\big), \\ C &= 2(a-b)\,X + \sum_{\text{gaps}} \big(2o + k(2e+a)\big), \end{aligned} \tag{3}$$

where X is the number of mismatches, so the wavefront computes score-maximising alignments. The cost also bounds the band in which the alignment must be stored. Each gapped letter contributes at least $2e + a$ to C, and an alignment that deviates by t diagonals from both the main and the end diagonal contains at least $2t$ gapped letters; hence only diagonals within $C/(2(2e + a))$ of them can contain an optimal alignment. For a 1 kbp pair at 5% divergence, this stores 137 diagonals instead of 1,000. Under a general substitution matrix, anti-diagonals are computed in 16-bit vector lanes while the scores provably fit (Wozniak, 1997). Local alignment first locates the end of the optimal alignment with a forward sweep, as in SSW (Zhao et al., 2013), and then extends a wavefront backwards from that end, stopping at the first cell that attains the maximal score; this avoids the correction pass that striped layouts may need when long alignments score highly (Farrar, 2007).

2.8 Validation

A differential fuzzer compares results for random pairs, cost models, modes, bands, cost limits and tie-breaking rules with a full-matrix reference implementation that shares no code with the library, checking validity, optimality, the symmetry of the two tie-breaking rules and agreement between batched and single calls. The WFA2-lib regression set is reproduced exactly, including all CIGAR strings under the rightmost tie-breaking rule. The seed heuristic is compared with the true remaining cost at every cell of random pairs; this check found and confirmed the correction of an implementation error that caused occasional overestimates (Supplementary Section S1).

3 Results

3.1 Experimental setup

All measurements were taken on an Intel Core i9-7900X (Skylake-X, AVX-512) with ten cores fixed at 3.3 GHz, turbo boost and hyper-threading disabled, and 91 GB of memory, running Ubuntu 26.04. dinara-align is implemented in Mojo (Modular Inc., 2026) and was compiled with Mojo 1.1.0 at commit `dc980d7`, and for gap-affine costs and batches at `289bcf0`, whose code those workloads share; all tools were compiled for the native instruction set. The exact aligners compared are A*PA2-full and A*PA2-simple (Groot Koerkamp, 2024), A*PA (Groot Koerkamp and Ivanov, 2024), Edlib (Šošić and Šikić, 2017), WFA and BiWFA (Marco-Sola et al., 2021, 2023), QuickEd (Doblas et al., 2025), BSAlign (Shao and Ruan, 2024), KSW2 (Li, 2018; Suzuki and Kasahara, 2018), parasail (Daily, 2016) and SSW (Zhao et al., 2013). Every cost was compared with those of every other tool, and on the A*PA2 datasets with the costs recorded in the published A*PA2 results; no discrepancies occurred.

We combined the protocols of A*PA2 (Groot Koerkamp, 2024) and QuickEd (Doblas et al., 2025). The datasets are the real datasets of A*PA2, nanopore reads of about 1, 10, 50 and 500 kbp, the last also against a reference with genetic variation, and SARS-CoV-2 genomes, and the simulated datasets of QuickEd, random sequences of 10 kbp, 100 kbp, 500 kbp and 1 Mbp with 1, 5, 10 and 20% uniformly distributed errors (Supplementary Table S5). Each dataset was sampled in a fixed random order to about 2 Mbp of sequence and at least four pairs, the same sample for every tool. One single-threaded job ran at a time, pinned to one core; each tool aligned the pairs of a sample once each, with traceback, within 30 s per sample, after which only the completed pairs are reported, and tools whose memory grows with the matrix or the square of the distance ran under an 8 GB limit. We report the mean and median wall-clock time per alignment, excluding input, the increase in peak resident memory during an alignment, measured with `getrusage` as in A*PA2, and the peak resident memory of the process, as in QuickEd. Gap-affine costs used the same samples and protocol. The other modes and the short-read batches were timed in-process as the faster of two passes over each workload.

3.2 Edit distance

On the real datasets, dinara-align had the lowest mean and median time among the exact aligners (Table 2; Supplementary Table S4). It aligned the 1 kbp reads in 23 µs, compared with 31 µs for WFA, required about two thirds of the time of A*PA2-simple on the 10 and 50 kbp reads (157 against 247 µs and 590 against 944 µs), and aligned the ultra-long reads in 118 and 156 ms, compared with 283 and 225 ms for A*PA2-full and 847 ms and 1.39 s for QuickEd.

On the simulated datasets (Fig. 1; Supplementary Table S4), dinara-align was the fastest at every length and error rate. On 100 kbp pairs it required 1.5, 5.0,

Table 2: Unit-cost alignment of the A*PA2 real datasets: mean time per alignment, traceback included. Superscripts give the number of pairs completed within 30 s where not all were; a dash marks a tool that completed none, o.o.m. one the 8 GB limit stopped.

Dataset	dinara-align	A*PA2-full	A*PA2-simple	A*PA	Edlib	BiWFA	WFA	QuickEd
ont-1k (0.8 kbp)	**23 µs**	81 µs	48 µs	612 µs	124 µs	44 µs	31 µs	75 µs
ont-10k (3.6 kbp)	**157 µs**	391 µs	247 µs	15.3 ms	1.09 ms	741 µs	599 µs	688 µs
ont-50k (9.6 kbp)	**590 µs**	1.27 ms	944 µs	263 ms	6.14 ms	7.43 ms	6.38 ms	2.31 ms
ont-500k (642 kbp)	**118 ms**	283 ms	687 ms	–	5.00 s	6.33 s	o.o.m.	847 ms
genvar (663 kbp)	**156 ms**	225 ms	511 ms	–	5.23 s	6.20 s	o.o.m.	1.39 s
SARS-CoV-2 (30 kbp)	**285 µs**	2.00 ms	728 µs	6.49 ms	8.18 ms	909 µs	987 µs	3.90 ms

7.2 and 16.0 ms at 1, 5, 10 and 20% errors, compared with 7.5, 8.5, 13.9 and 36.7 ms for A*PA2-full and 12.5, 24.0, 39.0 and 62.8 ms for QuickEd, and on 1 Mbp pairs at 20% errors 901 ms, compared with 2.02 s for A*PA2-full and 4.8 s for QuickEd. Its narrowest margin was on 10 kbp pairs at 5%, 269 against 288 µs for A*PA2-simple. Over the 22 datasets, on the pairs both completed, A*PA2-full took 1.3 to 22.3 times as long as dinara-align (2.5 in the geometric mean), A*PA2-simple 1.1 to 12.8 times (3.2) and QuickEd 3.0 to 22.5 times (6.7).

On the ultra-long reads, an alignment increased peak memory by at most 51 MB without and 37 MB with genetic variation, compared with 82 and 48 MB for A*PA2-full and 82 and 32 MB for A*PA2-simple; on 1 Mbp pairs at 20% errors, by at most 126 MB, compared with 184 and 192 MB for A*PA2-full and A*PA2-simple, the tile boundaries kept for the traceback staying within the default limit of 80 MB (Supplementary Section S1). QuickEd, which splits its traceback recursively above 16 MB, stayed near 25 MB on every dataset, and Edlib below 8 MB. On shorter inputs dinara-align generally used more memory than the most economical tool, and its runtime occupies about 13 MB, compared with 2 to 8 MB for the other tools.

3.3 Gap-affine costs, other modes and batches

At the WFA costs (4, 6, 2), dinara-align was 1.7 to 3.8 times faster than WFA and BiWFA on every dataset that any tool completed, 2.7 and 2.6 times in the geometric mean, and it completed more of the long pairs than any other tool: 8 of the 10 pairs of 100 kbp at 20% errors, against 4 for BiWFA (Table 3; Supplementary Table S3). No tool completed a pair of 1 Mbp beyond 1% errors within 30 s. In each of the 14 workloads covering local, overlap, infix, prefix, extension, two-piece and substitution-matrix alignment, it was at least as fast as the fastest alternative, by factors from 1.0 to 22 (Supplementary Table S2); in the one tie, infix alignment at (4, 6, 2), its time varies between 3.0 and 3.9 ms from build to build with the placement of one loop, for the same instructions.

Following the short-read case study of Accelign (Kallenborn et al., 2026), we computed global scores for 500,000 pairs of 148 bp reads with 1% error against reference segments of similar length (Table 4). Under gap-affine costs (mismatch 1, gap $2 + k$), dinara-align required 0.46 µs per pair on one core, compared with 2.12 µs for WFA2-lib, and 27 ms for the whole batch on ten cores, compared with 116 ms.

3.4 Effect of each technique

To measure what each technique contributes, dinara-align was built with one technique switched off at a time, everything else unchanged, and every build aligned samples of the A*PA2 real datasets and of the 100 kbp simulated pairs with traceback, the ultra-long reads ten times the usual sample, and scored the short-read batch. The builds alternated within each of three rounds, pinned to one core, and we report the median against the build with every technique on (Table 5). Because the Skylake-X penalises jumps that cross 32-byte boundaries, the position of a loop after a rebuild can move the running time by several percent; the alignment builds were therefore assembled with branches kept clear of those boundaries (`-mbranches-within-32B-boundaries`). A baseline built in this way from a different source file ran within 0.7% of the baseline on every dataset, and every build returned the same costs on every dataset. Constructing the neighbours of exact seed matches directly contributed most on the ultra-long reads, which without it took 9% and 16% longer; on 100 kbp pairs, which now take exact seeds on AVX-512, it made no difference. Interleaving two groups of bit-parallel words contributed 4.5 to 5% on the ultra-long reads and 11.5% on 100 kbp pairs at 20% errors, whose bands are widest; the regrouped recurrence, although it shortens a single word's step from 14 to 8 cycles, contributed 1 to 4%, because bands narrow enough to be bound by that step make up a small part of the work. Gathered diagonal transition contributed 13 to 27% where diagonal transition does most of the work, on the 1 kbp reads, the SARS-CoV-2 genomes and 100 kbp pairs at 1% errors. The agreement rule for extrapolated bounds mattered on the 10 kbp reads (22%) and the SARS-CoV-2 genomes (5%), and not on the ultra-long reads, where the seed heuristic sets the first bound. Without the certified band, the short-read batch took 2.9 and 3.2 times as long at affine and unit costs.

4 Discussion

The algorithms in dinara-align build on those of earlier exact aligners; the gains come from changes to how these algorithms use the processor. Two observations stand out. First, on long pairs the techniques that reduce

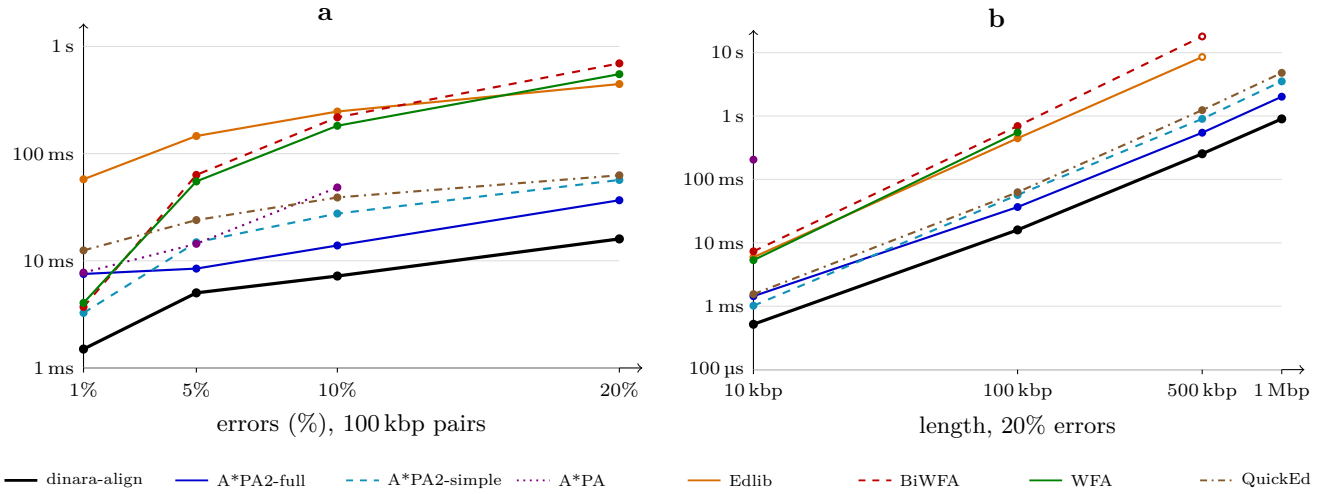


Figure 1: Unit-cost alignment of QuickEd's simulated pairs, mean time per alignment on a logarithmic scale. (a) 100 kbp pairs by error rate. (b) Pairs at 20% errors by length (5% in the interactive version); lines end where a tool completed no pair within 30 s, and hollow points mark tools that completed only some pairs. dinara-align had the lowest mean time at every length and error rate.

Table 3: Gap-affine alignment at costs (4, 6, 2): mean time per alignment with a 30 s budget per sample. Superscripts give the number of pairs completed where not all were; o.o.m. marks a tool the 8 GB limit stopped.

Dataset	dinara-align	WFA	BiWFA	KSW2	BSAlign
ont-1k	**112 µs**	186 µs	304 µs	1.10 ms	336 µs
ont-10k	**1.71 ms**	4.80 ms	5.84 ms	29.8 ms	7.84 ms
ont-50k	**22.0 ms**	63.5 ms	54.9 ms	343 ms$^{81/104}$	87.0 ms
SARS-CoV-2	**737 µs**	2.06 ms	1.79 ms	88.4 ms	341 ms
10 kbp, 5%	**2.32 ms**	7.05 ms	8.72 ms	97.5 ms	37.0 ms
10 kbp, 20%	**20.6 ms**	65.2 ms	75.2 ms	374 ms$^{81/99}$	37.4 ms
100 kbp, 1%	**14.0 ms**	39.7 ms	39.0 ms	2.44 s	o.o.m.
100 kbp, 5%	**337 ms**	811 ms	715 ms	9.25 s$^{3/9}$	o.o.m.

work outside the inner loop contributed more than those that speed it up: constructing seed neighbours directly saved up to 16%, against 4.5 to 11.5% for interleaving bit-parallel groups and 1 to 4% for the regrouped recurrence (Table 5). Second, thresholds that encode a cost model go stale as the code around them changes. A tile width for narrow bands, kept from the first version, and the length from which seeds match with one edit cost 10 kbp pairs at 5% a fifth and 100 kbp pairs at 10% a third of their running time, and a projection that handed divergent gap-affine pairs to a full sweep slowed nanopore reads by half, until every threshold was measured again by halving and doubling it (Supplementary Section S1). Such a sweep belongs in the evaluation of any aligner whose choice of algorithm depends on fixed thresholds. Neither the regrouped recurrence nor the per-lane band certificate depends on the implementation, and both apply directly to other bit-parallel and inter-sequence aligners.

The study has several limitations. First, the evaluation covers a single x86 processor with AVX-512. The AVX2 configuration was measured only for its inexact-seed threshold, built for Haswell on the same machine (20 and 30% equal within 0.5%, 40% up to 6% slower), and the NEON configuration, which the library also supports, is not evaluated here. Second, some thresholds were chosen using workloads that also appear in the evaluation; in particular, the AVX-512 thresholds for inexact seeds, the share of chained seeds (lowered from 40 to 20%) and the length from which they are used (raised from 64 to 128 kbp), were set after measurements on uniform 100 kbp pairs like those of Fig. 1a, so results on those pairs may overstate performance on unseen data. Third, as in the A*PA2 protocol, each pair was aligned once, and on the Skylake-X some running times varied by up to one third between builds that did not change the code involved, which we attribute to code alignment effects in the instruction decoder; the ablation controls for this, the comparison with other tools does not. Fourth, the ablation switches off one technique at a time against the full configuration, so it does not separate interactions between techniques. Finally, release builds target the baseline processor of each platform, so the AVX-512 code paths require a build for the host processor.

Future work includes speeding up seed construction, reusing computation across band-doubling rounds as in the incremental doubling of A*PA2, measurements on further processors, and GPU implementations of the same algorithms.

Funding

This work received no specific funding.

Table 4: Global scores of 500,000 short-read pairs; each time is the whole call.

Costs	Threads	dinara-align	WFA2-lib	KSW2	parasail	Edlib
Affine (1, 2, 1)	1	**230 ms**	1.06 s	10.9 s	21.7 s	–
Unit	1	**212 ms**	450 ms	–	–	2.00 s
Affine (1, 2, 1)	10	**27 ms**	116 ms	1.21 s	2.32 s	–
Unit	10	**29 ms**	64 ms	–	–	216 ms

Table 5: Effect of each technique: median time with all techniques on (ms per alignment; per batch of 500,000 pairs for the short reads), and the change when one technique is switched off, over three alternating rounds on the Skylake-X at commit `dc980d7`, with branches kept clear of 32-byte boundaries. Every configuration returned the same costs.

Dataset	All on	Regrouped recurrence	Paired groups	Gathers	Agreement rule	Seed neighbours	Certified band
ont-500k	98.8	+1.1%	**+4.5%**	+0.5%	−0.2%	**+8.9%**	
ont-500k-genvar	136.0	+1.9%	**+5.0%**	+0.4%	0.0%	**+16.4%**	
ont-1k	0.021	+0.2%	+0.3%	**+14.9%**	0.0%	+0.7%	
ont-10k	0.145	−0.6%	+1.2%	**+6.4%**	**+22.2%**	+0.1%	
SARS-CoV-2	0.197	−0.9%	+0.2%	**+13.4%**	**+4.9%**	−0.5%	
100 kbp, 1%	1.10	−1.4%	+0.4%	**+26.5%**	−0.1%	−1.0%	
100 kbp, 5%	4.61	−0.6%	−0.2%	+0.1%	0.0%	+0.2%	
100 kbp, 10%	6.72	+0.3%	+3.4%	+1.9%	+0.5%	−0.2%	
100 kbp, 20%	15.0	**+4.1%**	**+11.5%**	+3.9%	−2.9%	0.0%	
Short reads, affine	232						**+188%**
Short reads, unit	213						**+215%**

Conflict of interest

None declared.

Data availability

The software, benchmark harness and the commits of every compared tool are available at `https://github.com/kokyriakidis/dinara-align`. The benchmarks are reproduced by `pixi run bench` (unit and `-affine` costs), `bench-modes`, `bench-batch` and `bench-ablation`.

References

Jeff Daily. Parasail: SIMD C library for global, semi-global, and local pairwise sequence alignments. *BMC Bioinformatics*, 17:81, 2016. doi: 10.1186/s12859-016-0930-z.

Max Doblas, Oscar Lostes-Cazorla, Quim Aguado-Puig, Nick Cebry, Pau Fontova-Musté, Christopher Batten, Santiago Marco-Sola, and Miquel Moretó. QuickEd: high-performance exact sequence alignment based on bound-and-align. *Bioinformatics*, 41(3):btaf112, 2025. doi: 10.1093/bioinformatics/btaf112.

Jordan M. Eizenga and Benedict Paten. Improving the time and space complexity of the WFA algorithm and generalizing its scoring. bioRxiv, 2022.

Michael Farrar. Striped Smith–Waterman speeds database searches six times over other SIMD implementations. *Bioinformatics*, 23(2):156–161, 2007. doi: 10.1093/bioinformatics/btl582.

Daichi Fujiki, Shunhao Wu, Nathan Ozog, Kush Goliya, David Blaauw, Satish Narayanasamy, and Reetuparna Das. SeedEx: A genome sequencing accelerator for optimal alignments in subminimal space. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 937–950, 2020. doi: 10.1109/MICRO50266.2020.00080.

Jean-François Gibrat. A short note on dynamic programming in a band. *BMC Bioinformatics*, 19:226, 2018. doi: 10.1186/s12859-018-2228-9.

Osamu Gotoh. An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3):705–708, 1982. doi: 10.1016/0022-2836(82)90398-9.

Ragnar Groot Koerkamp. A*PA2: Up to 19× faster exact global alignment. In Solon P. Pissis and Wing-Kin Sung, editors, *24th International Workshop on Algorithms in Bioinformatics (WABI 2024)*, volume 312 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 17:1–17:25. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. doi: 10.4230/LIPIcs.WABI.2024.17.

Ragnar Groot Koerkamp and Pesho Ivanov. Exact global alignment using A* with chaining seed heuristic and match pruning. *Bioinformatics*, 40(3):btae032, 2024. doi: 10.1093/bioinformatics/btae032.

Felix Kallenborn, Fawaz Dabbaghie, Martin Steinegger, and Bertil Schmidt. Accelign: a GPU-based library for accelerating pairwise sequence alignment. *BMC Bioinformatics*, 27:137, 2026. doi: 10.1186/s12859-026-06521-0.

Heng Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018. doi: 10.1093/bioinformatics/bty191.

Daniel Liu and Martin Steinegger. Block aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices. *Bioinformatics*,

39(8):btad487, 2023. doi: 10.1093/bioinformatics/btad487.

Santiago Marco-Sola, Juan Carlos Moure, Miquel Moreto, and Antonio Espinosa. Fast gap-affine pairwise alignment using the wavefront algorithm. *Bioinformatics*, 37(4):456–463, 2021. doi: 10.1093/bioinformatics/btaa777.

Santiago Marco-Sola, Jordan M. Eizenga, Andrea Guarracino, Benedict Paten, Erik Garrison, and Miquel Moreto. Optimal gap-affine alignment in $O(s)$ space. *Bioinformatics*, 39(2):btad074, 2023. doi: 10.1093/bioinformatics/btad074.

Modular Inc. Mojo programming language. `https://docs.modular.com/mojo/`, 2026.

Eugene W. Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1(1–4):251–266, 1986. doi: 10.1007/BF01840446.

Gene Myers. A fast bit-vector algorithm for approximate string matching based on dynamic programming. *Journal of the ACM*, 46(3):395–415, 1999. doi: 10.1145/316542.316550.

René Rahn, Stefan Budach, Pascal Costanza, Marcel Ehrhardt, Jonny Hancox, and Knut Reinert. Generic accelerated sequence alignment in SeqAn using vectorization and multi-threading. *Bioinformatics*, 34 (20):3437–3445, 2018. doi: 10.1093/bioinformatics/bty380.

Torbjørn Rognes. Faster Smith-Waterman database searches with inter-sequence SIMD parallelisation. *BMC Bioinformatics*, 12:221, 2011. doi: 10.1186/1471-2105-12-221.

Haojing Shao and Jue Ruan. BSAlign: a library for nucleotide sequence alignment. *Genomics, Proteomics & Bioinformatics*, 22(2):qzae025, 2024. doi: 10.1093/gpbjnl/qzae025.

Martin Šošić and Mile Šikić. Edlib: a C/C++ library for fast, exact sequence alignment using edit distance. *Bioinformatics*, 33(9):1394–1395, 2017. doi: 10.1093/bioinformatics/btw753.

Hajime Suzuki and Masahiro Kasahara. Introducing difference recurrence relations for faster semi-global alignment of long sequences. *BMC Bioinformatics*, 19 (Suppl 1):45, 2018. doi: 10.1186/s12859-018-2014-8.

Esko Ukkonen. Algorithms for approximate string matching. *Information and Control*, 64(1–3):100–118, 1985. doi: 10.1016/S0019-9958(85)80046-2.

Andrzej Wozniak. Using video-oriented instructions to speed up sequence comparison. *Computer Applications in the Biosciences (CABIOS)*, 13(2):145–150, 1997. doi: 10.1093/bioinformatics/13.2.145.

Mengyao Zhao, Wan-Ping Lee, Erik P. Garrison, and Gabor T. Marth. SSW library: An SIMD Smith-Waterman C/C++ library for use in genomic applications. *PLoS ONE*, 8(12):e82138, 2013. doi: 10.1371/journal.pone.0082138.